

Programozás komplex beadandó feladat

Készítette: Fegyver Dávid Bátor

- Neptun-azonosító: **MKLH4B**
 - E-mail: root@davidfegyver.hu
 - Kurzuskód: **IT-18PROGEG**
 - Gyakorlatvezető neve: **Németh Gergely Zétény**
 - Elkészítve: **2025. január 02.**
-

Felhasználói dokumentáció

Feladat: Legmelegebb települések

- A meteorológiai intézet az ország N településére adott M napos időjárás előrejelzést, az adott településen az adott napra várt legmagasabb hőmérsékletet.
- A program megadja azokat a településeket, ahol a legmagasabb hőmérséklet várható.

Futási környezet

IBM PC, exe futtatására alkalmas, 64-bites operációs rendszer (pl. Windows 11). Nem igényel egeret.

Használat

A program indítása

A program **MKLH4B_LegmelegebbTelepulesek_Minta.exe** és **MKLH4B_LegmelegebbTelepulesek_Speci.exe** neveken található a tömörített állományban

A program használata billentyűzetről való bevitel esetén

Az .exe kiterjesztésű fájlok valamelyikének elindításával a program az adatokat a **billentyűzetről** olvassa be a következő sorrendben:

#	Adat	Magyarázat
1.	Települések száma (n)	Pozitív egész szám
2.	Napok száma (m)	Pozitív egész szám
3.	1. település adatai	Pozitív egészek szóközzel elválasztva
4.	2. település adatai	
...	...	
	n. település adatai	

A program használata fájlból való bevitel esetén

Lehetőségünk van az adatokat fájlban is megadni. Ekkor a programot powershellben a következőképpen kell indítani, feltételezve, hogy a bemeneti fájlok mellette helyezkednek el:

```
Get-Content .\be.txt | .\MKLH4B_LegmelegebbTelepuések_Speci.exe
2 2 3
```

A fájl felépítésének a következő formai követelményei vannak:

A fájl első sorában a települések száma ($1 \leq N \leq 1000$) és a napok száma ($1 \leq M \leq 1000$) van. Az ezt követő N sorban az egyes napokra jósolt M hőmérséklet értéke található ($-50 \leq H_{i,j} \leq 50$).

Például:

```
4 3
30 25 20
10 10 10
15 30 12
30 30 30
```

A program kimenete

A program megadja azokat a településeket, ahol a legmagasabb hőmérséklet várható.

Minta bemenet és kimenet

```
=== Legmelegebb települések ===
Kérem a települések számát (n): 3
Kérem a napok számát (m): 5
Kérem a hőmérsékleteket (5 db szám soronként):
1. település adatai: 10 15 12 10 10
2. település adatai: 11 11 11 11 20
3. település adatai: 12 16 16 16 20
=== Eredmény ===
Ennyi település érte el a legmagasabb hőmérsékletet: 2
A települések sorszámai: 2 3
Kérem, nyomjon ENTER-t a kilépéshez!
|
```

Hibalehetőségek

Az egyes bemeneti adatokat a fenti mintának megfelelően kell megadni. Hiba, ha bármelyik megadandó adat nem természetes szám. Hiba esetén a program azzal jelzi a hibát, hogy újra kérdezi azt.

Mintafutás hibás bemeneti adatok esetén:

```
=== Legmelegebb települések ===
Kérem a települések számát (n): 0
Hiba: A települések számának pozitív egésznek kell lennie!
Kérem a települések számát (n): 6
Kérem a napok számát (m): 7
Kérem a hőmérsékleteket (7 db szám soronként):
1. település adatai: meow
Hiba: Pontosan 7 számot kell megadni!
1. település adatai: pikachu
Hiba: Pontosan 7 számot kell megadni!
1. település adatai: |
```

Fejlesztői dokumentáció

Tervezés

Specifikáció

[speci_online](#)

```
Be: n ∈ N, m ∈ N, homletek ∈ N[1..n, 1..m]
Ki: db ∈ N, maxhomletek ∈ N[]
```

```

Fv: telepulesMax:N→N,
    telepulesMax(i)=MAX(j=1..m, homletek[i][j]).maxért
Fv: telepulesMaxOsszMaxE: N→L,
    telepulesMaxOsszMaxE(i) = MAX(j=1..n, telepulesMax(j)).maxért = telepulesMax(i)
Ef:  $1 \leq n \leq 1000$  és  $1 \leq m \leq 1000$  és  $\forall i \in [1..n]: (\exists j \in [1..m]: (-50 \leq \text{homletek}[i,j] \leq 50))$ 
Uf: (db,maxhomletek)=KIVÁLOGAT(i=1..n, telepulesMaxOsszMaxE(i), i)

```

Visszavezetési táblázatok

A *telepulesMax(i)* függvényben található maximumkeresési minta visszavezetése

```

i ~ j
e ~ 1
u ~ m
f(i) ~ homletek[i][j]

```

A *telepulesMaxOsszMaxE(i)* függvényben található maximumkeresési minta visszavezetése

```

i ~ j
e ~ 1
u ~ n
f(i) ~ telepulesMax(j)

```

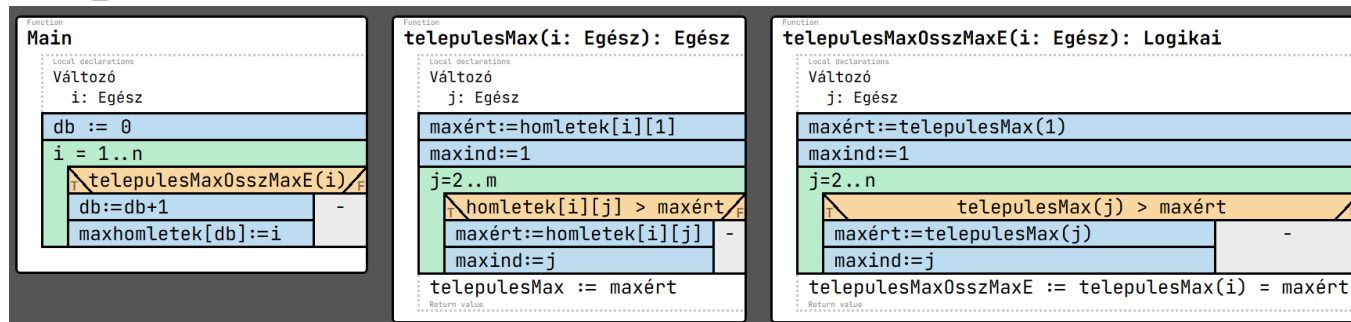
Az *utófeltételben* található kiválogatási minta visszavezetése

```

db ~ db
y ~ maxhomletek
i ~ i
e ~ 1
u ~ n
T(i) ~ telepulesMaxOsszMaxE(i)
f(i) ~ i

```

Algoritmus



Fejlesztői környezet

IBM PC, exe futtatására alkalmas operációs rendszer (pl. Windows 11 Home). Visual Studio 2022 (Version 17.2.3) fejlesztői környezet.

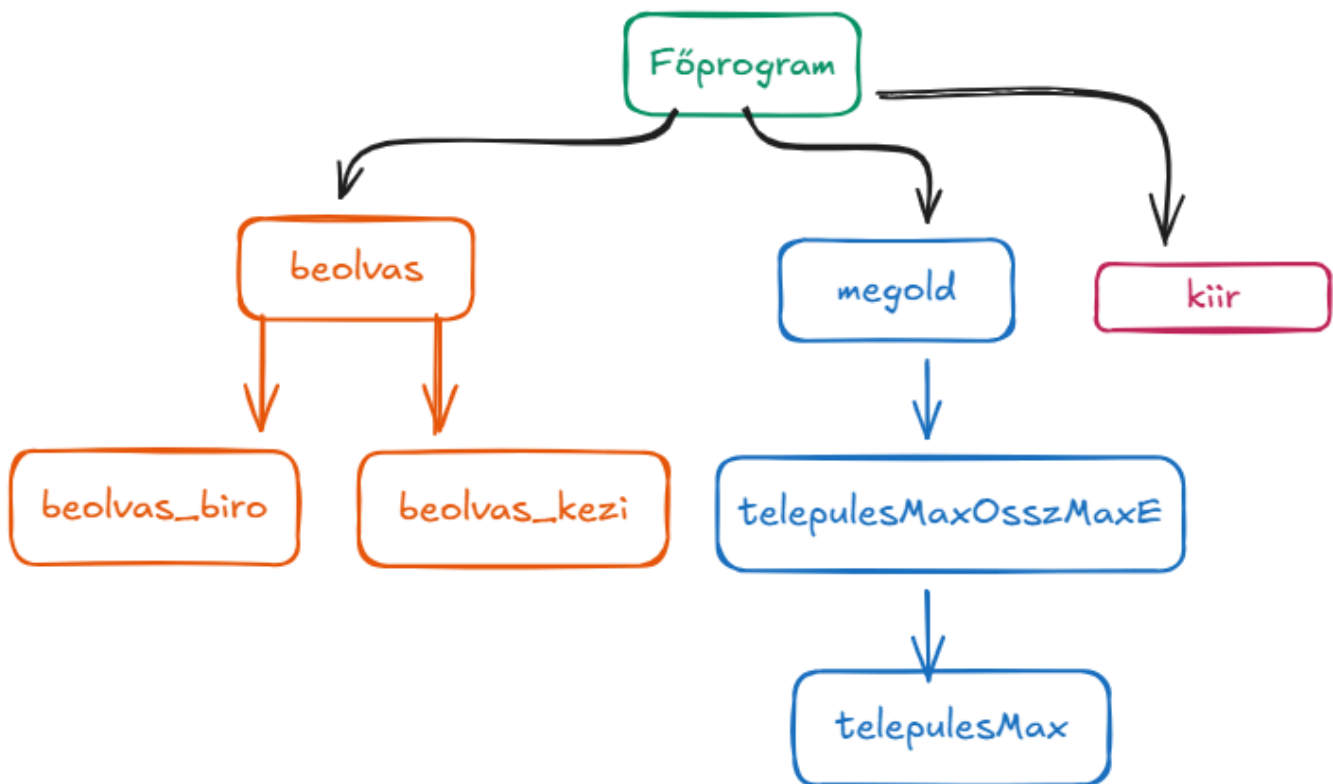
Forráskód

A teljes fejlesztői anyag –kicsomagolás után– az MKLH4B nevű könyvtárban található meg. A fejlesztés során használt könyvtár-struktúra:

Állomány	Magyarázat
MKLH4B\MKLH4B_LegmelegebbTelepulesek_Minta.exe	Futtatható fájl
MKLH4B\MKLH4B_LegmelegebbTelepulesek_Speci.exe	Futtatható fájl
MKLH4B\MKLH4B_LegmelegebbTelepulesek_Minta.cs	C# forráskód
MKLH4B\MKLH4B_LegmelegebbTelepulesek_Speci.cs	C# forráskód
Mintak.cs	C# forráskód, Programozási minták könyvtára
feladat.pdf	A feladat
tesztek*	Teszt bemeneti fájlok
MKLH4B.pdf	Dokumentáció (ez a fájl)

Megoldás

Függvényszerkezet



Kód

A *MKLH4B_LegmelegebbTelepulesek_Speci.cs* fájl tartalma:

```
/*
Készítette: Fegyver Dávid Bátor
Neptun-azonosító: MKLH4B
E-mail: root@davidfegyver.hu
Feladat: Legmelegebb települések
*/
using System;

namespace LegmelegebbTelepulesek
{
    class Program
    {
        static int n;
        static int m;
        static int[,] homletek;

        static int db;
        static int[] maxhomletek;
        static int[] telepulesMaxCache;

        static void Main()
        {
            beolvas();
        }
    }
}
```

```

        megold();

        kiir();
    }

    static void beolvas()
    {
        bool biroE = Console.IsInputRedirected;
        if (biroE)
        {
            homletek = beolvas_biro();
        }
        else
        {
            homletek = beolvas_kezi();
        }

        telepulesMaxCache = new int[n];
        for (int i = 0; i < n; i++) telepulesMaxCache[i] = -1;
    }

    static int[,] beolvas_biro()
    {
        string eSor = Console.ReadLine();
        string[] eAdatok = eSor.Split(" ");

        n = int.Parse(eAdatok[0]);
        m = int.Parse(eAdatok[1]);

        int[,] mat = new int[n, m];

        for (int i = 0; i < n; i++)
        {
            string sor = Console.ReadLine();

            string[] adatok = sor.Split(" ");
            for (int j = 0; j < m; j++)
            {
                mat[i, j] = int.Parse(adatok[j]);
            }
        }

        return mat;
    }

    static int[,] beolvas_kezi()
    {
        Console.WriteLine("≡≡≡ Legmelegebb települések ≡≡≡");
    }

```

```

bool siker = false;
do
{
    Console.Write("Kérem a települések számát (n): ");
    string be = Console.ReadLine();

    if (int.TryParse(be, out n) && n > 0)
    {
        siker = true;
    }
    else
    {
        Console.WriteLine("Hiba: A települések számának pozitív
egésznek kell lennie!");
        siker = false;
    }
}
while (!siker);

siker = false;
do
{
    Console.Write("Kérem a napok számát (m): ");
    string be = Console.ReadLine();

    if (int.TryParse(be, out m) && m > 0)
    {
        siker = true;
    }
    else
    {
        Console.WriteLine("Hiba: A napok számának pozitív egésznek
kell lennie!");
        siker = false;
    }
}
while (!siker);

int[, ] mat = new int[n, m];

Console.WriteLine($"Kérem a hőmérsékleteket ({m} db szám
soronként):");

for (int i = 0; i < n; i++)
{
    siker = false;
    do
    {
        Console.Write($"{i + 1}. település adatai: ");
        string sor = Console.ReadLine();
    }
    while (!siker);

    mat[i, 0] = sor;
}

```

```

        string[] adatok = sor.Split(" ");

        if (adatok.Length != m)
        {
            Console.WriteLine($"Hiba: Pontosán {m} számot kell
megadni!");

            siker = false;
        }
        else
        {
            bool mindenJo = true;
            for (int j = 0; j < m; j++)
            {
                if (!int.TryParse(adatok[j], out mat[i, j]))
                {
                    Console.WriteLine($"Hiba: A bemeneted tartalmaz
nem érvényes számot!");

                    mindenJo = false;
                    break;
                }
            }
            siker = mindenJo;
        }
    }
    while (!siker);
}
return mat;
}

static void megold()
{
    db = 0;
    maxhomletek = new int[n];
    for (int i = 0; i < n; i++)
    {
        if (telepulesMaxOsszMaxE(i))
        {
            maxhomletek[db] = i;
            db++;
        }
    }
}

static int telepulesMax(int i)
{
    if (telepulesMaxCache[i] != -1) return telepulesMaxCache[i];

    int maxért = homletek[i, 0];
    int maxind = 0;

```

```

        for (int j = 1; j < m; j++)
        {
            if (homletek[i, j] > maxért)
            {
                maxért = homletek[i, j];
                maxind = j;
            }
        }

        telepulesMaxCache[i] = maxért;

        return maxért;
    }

    static bool telepulesMaxOsszMaxE(int i)
    {
        int maxért = telepulesMax(0);
        int maxind = 0;
        for (int j = 1; j < n; j++)
        {
            int Jmax = telepulesMax(j);
            if (Jmax > maxért)
            {
                maxért = Jmax;
                maxind = j;
            }
        }
        return telepulesMax(i) == maxért;
    }

    static void kiir()
    {
        bool biroE = Console.IsInputRedirected;

        if (!biroE) Console.WriteLine("≡ Eredmény ≡");

        if (!biroE) Console.Write("Ennyi település érte el a legmagasabb  
hőmérsékletet: ");
        Console.Write(db);
        Console.Write(" ");
        if (!biroE) Console.Write("\nA települések sorszámai: ");

        for (int i = 0; i < db; i++)
        {
            Console.Write(maxhomletek[i] + 1);
            if (i < db - 1) Console.Write(" ");
        }
        Console.WriteLine();
    }

```

```

        if (!biroE)
        {
            Console.WriteLine("Kérem, nyomjon ENTER-t a kilépéshez! ");
            Console.ReadLine();
        }
    }
}
}

```

Kód (magas szintű függvényekkel)

A *MKLH4B_LegmelegebbTelepulesek_Minta.cs* fájl tartalma:

```

/*
Készítette: Fegyver Dávid Bátor
Neptun-azonosító: MKLH4B
E-mail: root@davidfegyver.hu
Feladat: Legmelegebb települések
*/
using System;
using magas_szintű_mintamegvalósítások;

namespace LegmelegebbTelepulesek
{
    class Program
    {
        static int n;
        static int m;
        static int[,] homletek;

        static int db;
        static int[] maxhomletek;

        static void Main()
        {
            beolvas();

            megold();

            kiir();
        }

        static void beolvas()
        {

```

```

        bool biroE = Console.IsInputRedirected;
        if (biroE)
        {
            homletek = beolvas_biro();
        }
        else
        {
            homletek = beolvas_kezi();
        }
    }

    static int[,] beolvas_biro()
    {
        string eSor = Console.ReadLine();
        string[] eAdatok = eSor.Split(" ");

        n = int.Parse(eAdatok[0]);
        m = int.Parse(eAdatok[1]);

        int[,] mat = new int[n, m];

        for (int i = 0; i < n; i++)
        {
            string sor = Console.ReadLine();

            string[] adatok = sor.Split(" ");
            for (int j = 0; j < m; j++)
            {
                mat[i, j] = int.Parse(adatok[j]);
            }
        }

        return mat;
    }

    static int[,] beolvas_kezi()
    {
        Console.WriteLine("≡≡≡ Legmelegebb települések ≡≡≡");

        bool siker = false;
        do
        {
            Console.Write("Kérem a települések számát (n): ");
            string be = Console.ReadLine();

            if (int.TryParse(be, out n) && n > 0)
            {
                siker = true;
            }
        }
        else
    }

```

```

        {
            Console.WriteLine("Hiba: A települések számának pozitív
egésznek kell lennie!");
            siker = false;
        }
    }
    while (!siker);

    siker = false;
    do
    {
        Console.Write("Kérem a napok számát (m): ");
        string be = Console.ReadLine();

        if (int.TryParse(be, out m) && m > 0)
        {
            siker = true;
        }
        else
        {
            Console.WriteLine("Hiba: A napok számának pozitív egésznek
kell lennie!");
            siker = false;
        }
    }
    while (!siker);

    int[, ] mat = new int[n, m];

    Console.WriteLine($"Kérem a hőmérsékleteket ({m} db szám
soronként):");

    for (int i = 0; i < n; i++)
    {
        siker = false;
        do
        {
            Console.Write($"{i + 1}. település adatai: ");
            string sor = Console.ReadLine();

            string[] adatok = sor.Split(" ");

            if (adatok.Length != m)
            {
                Console.WriteLine($"Hiba: Pontosan {m} számot kell
megadni!");
                siker = false;
            }
            else
            {

```

```

        bool mindenJo = true;
        for (int j = 0; j < m; j++)
        {
            if (!int.TryParse(adatok[j], out mat[i, j]))
            {
                Console.WriteLine($"Hiba: A bemeneted tartalmaz
nem érvényes számot!");

                mindenJo = false;
                break;
            }
        }
        siker = mindenJo;
    }
    while (!siker);
}
return mat;
}

static void megold()
{
    maxhomletek = Mintak.Kivalogat(0, n - 1,
        i => telepulesMaxOsszMaxE(i),
        i => i
    );

    db = maxhomletek.Length;
}
static int telepulesMax(int i)
{
    return Mintak.Max(0, m - 1, j => homletek[i, j]).maxert;
}

static bool telepulesMaxOsszMaxE(int i)
{
    return telepulesMax(i) == Mintak.Max(0, n - 1, j =>
telepulesMax(j)).maxert;
}

static void kiir()
{
    bool biroE = Console.IsInputRedirected;

    if (!biroE) Console.WriteLine("≡≡≡ Eredmény ≡≡≡");

    if (!biroE) Console.WriteLine("Ennyi település érte el a legmagasabb
hőmérsékletet: ");
    Console.WriteLine(db);
    Console.WriteLine(" ");
    if (!biroE) Console.WriteLine("\nA települések sorszámai: ");
}

```

```
        for (int i = 0; i < db; i++)
        {
            Console.Write(maxhomletek[i] + 1);
            if (i < db - 1) Console.Write(" ");
        }
        Console.WriteLine();

        if (!biroE)
        {
            Console.WriteLine("Kérem, nyomjon ENTER-t a kilépéshez! ");
            Console.ReadLine();
        }
    }
}
```

Tesztelés

Automatikus tesztek (Bíró)

Összpont: 100/

Teszt#	Pont	...Verdikt...	futási idő
1.1	4/4	Helyes	0.032 sec
2.1	4/4	Helyes	0.034 sec
3.1	4/4	Helyes	0.034 sec
4.1	4/4	Helyes	0.033 sec
5.1	4/4	Helyes	0.032 sec
6.1	5/5	Helyes	0.032 sec
7.1	5/5	Helyes	0.032 sec
8.1	5/5	Helyes	0.032 sec
9.1	5/5	Helyes	0.032 sec
10.1	5/5	Helyes	0.033 sec
11.1	5/5	Helyes	0.033 sec
12.1	5/5	Helyes	0.033 sec
13.1	5/5	Helyes	0.036 sec
14.1	5/5	Helyes	0.034 sec
15.1	5/5	Helyes	0.035 sec
16.1	6/6	Helyes	0.068 sec
17.1	6/6	Helyes	0.067 sec
18.1	6/6	Helyes	0.060 sec
19.1	6/6	Helyes	0.123 sec
20.1	6/6	Helyes	0.317 sec

Beadva: 2026-01-03 00:51:37.0

Automatikus tesztek (Bíró, magas szintű függvényekkel)

Összpont: 76/

Teszt#	Pont	...Verdikt...	futási idő
1.1	4/4	Helyes	0.033 sec
2.1	4/4	Helyes	0.034 sec
3.1	4/4	Helyes	0.033 sec
4.1	4/4	Helyes	0.036 sec
5.1	4/4	Helyes	0.034 sec
6.1	5/5	Helyes	0.036 sec
7.1	5/5	Helyes	0.032 sec
8.1	5/5	Helyes	0.034 sec
9.1	5/5	Helyes	0.034 sec
10.1	5/5	Helyes	0.036 sec
11.1	5/5	Helyes	0.036 sec
12.1	5/5	Helyes	0.042 sec
13.1	5/5	Helyes	0.051 sec
14.1	5/5	Helyes	0.043 sec
15.1	5/5	Helyes	0.067 sec
16.1	0/6	Időlimit túllépés	1.001 sec
17.1	6/6	Helyes	0.315 sec
18.1	0/6	Időlimit túllépés	0.787 sec
19.1	0/6	Időlimit túllépés	1.002 sec
20.1	0/6	Időlimit túllépés	1.008 sec

Beadva: 2026-01-03 01:44:43.0

Érvényes tesztesetek

1. teszteset: be1.txt

Be:

```
3 5
10 15 12 10 10
11 11 11 11 20
12 16 16 16 20
```

Ki:

```
2 2 3
```

2 teszt eset: be2.txt

Be:

```
3 4
10 12 15 11
20 22 19 21
15 16 10 10
```

Ki:

```
1 2
```

3. teszt eset: be3.txt

Be:

```
4 3
30 25 20
10 10 10
15 30 12
30 30 30
```

Ki:

```
3 1 3 4
```

4. teszt eset: be4.txt

Be:

```
2 5
-10 -12 -15 -20 -10
```

```
-5 -8 -6 -5 -7
```

Ki:

```
1 2
```

5. teszt eset: be5.txt

Be:

```
1 1
100
```

Ki:

```
1 1
```

6. teszt eset: be6.txt

Be:

```
3 3
10 10 10
10 10 10
10 10 10
```

Ki:

```
3 1 2 3
```

Érvénytelen tesztesetek

Billentyűzetes bevitel esetén

1. teszt eset

Be: negatív szám

```
-6 7
```

2. teszt eset

Be: szöveg

sixseven